Name:_____ Date:_____

Physics 0525: Lab 13

Arduino III

Equipment:
Oscilloscope, DMM, 10 V power supply, Arduino Uno, computer with Arduino IDE, thermocouple amplifier (MAX6675) breakout board and 2N3725 bipolar junction transistor from the Arduino Kit.

Preliminaries:
In the previous two labs we mentioned several times that microcontrollers are designed to control environments and processes by reading sensors and operating actuators. However, up to this point you have only controlled an LED, which is not very impressive. In this lab, you will tackle a more difficult and practical control task.

Power Supplies
A typical laboratory DC power supply is shown in Figure 1. You will notice that it has three terminals: +, −, and ⏚ (ground). The voltage is generated between the + and − terminals just like in a battery. These two terminals are floating relative to the ground terminal (i.e. they not connected internally).

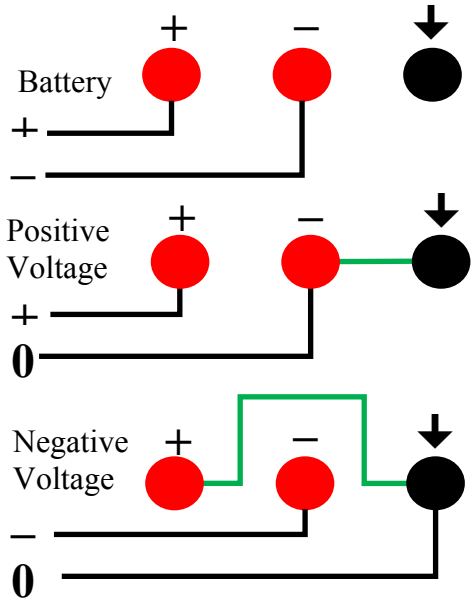


Figure 1. Left: Front panel of a single-output DC power supply (left) and Right: Jumper wire (green) used to configure power supply output as a floating (top) or ground referenced power supply: positive supply (middle) and negative supply (bottom).

Name:_____ Date:_____

With an additional jumper wire, the power supply can be configured to be ground referenced: either a positive power supply (with − and GND terminals connected), or a negative power supply (with + and GND terminals connected) as shown on the right side of Figure 1. Adding a ground-referenced power supply to a circuit will introduce Earth ground at the point where the power supply GND terminal is connected.

You will use the floating power supply configuration later with the Arduino.

Name:_____ Date:_____

Part 1: Controlling Large Currents

The Arduino can supply only a small current ~50 mA and voltage ~5 V. This may not be enough to drive the load you want to power. For example, in the next part of this experiment, you will be controlling the current through a power resistor which is used as a heater. Instead of trying to drive the heater directly with the Arduino output, we use the Arduino to control a second device that acts as a switch for the larger current and voltage supplied by an external power supply. The switch could be a relay or a transistor. Here you will use a high-power bipolar transistor (2N3725) as our switch. The advantage of using a transistor over a relay is that it can be switched on and off very quickly, allowing us to use a PWM output of the Arduino. The pin assignments for the 2N3725 are given in Figure 2.

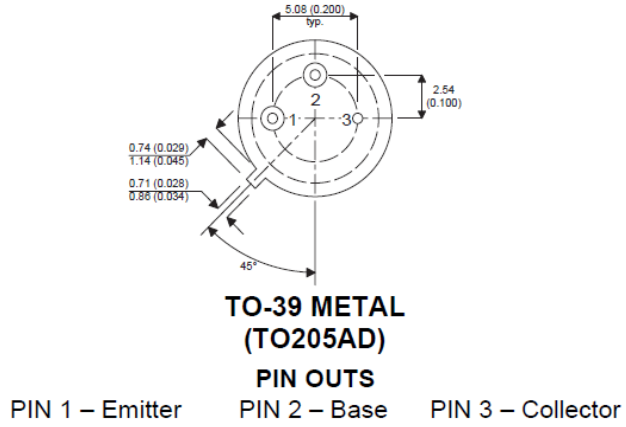


Figure 2. Pin assignments for the 2N3725 (bottom view).

The circuit to use for this part is shown in Figure 3. We use the Arduino output (pin 13) to drive the base of the transistor, which then passes a larger current through the collector to ground, sinking current through the load (in this case two LEDs). The 180 Ω resistor at the base of the transistor limits the current into the base, and the 330 Ω resistors limit the current through the LEDs. The LEDs are powered by an external 10 V floating power supply. The ground is provided by the Arduino.

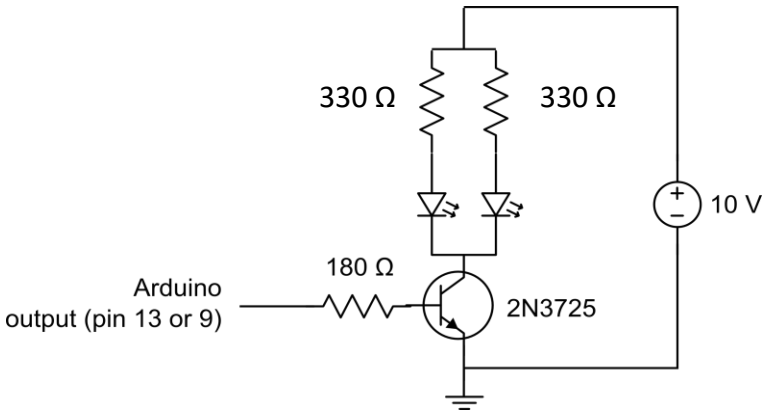


Figure 3. Using a transistor to control larger current.

Name:_____ Date:_____

WARNING: Make sure the lead of the 180 Ω base resistor does not touch the transistor case otherwise it will short the base and collector together which could damage your computer!

Blink Control of Large Currents
To flash the LEDs you can use the **Blink** sketch found in the Examples folder

```
File > Examples > 1.Basics > Blink
```

Measure the voltage across one of the 330Ω resistors when the LEDs are turned on and calculate the current drawn by a single LED in the circuit. To do this measurement, connect the jumper wire from the 180 Ω base resistor to +5V instead of pin 13.

6.52V

I_{LED} = 0.02 [A] $\frac{6.52V}{330\Omega} \Rightarrow 0.02A$

How many parallel LEDs could you drive with an Arduino Digital I/O pin?

$0.05/0.02 \simeq 2$ LEDs

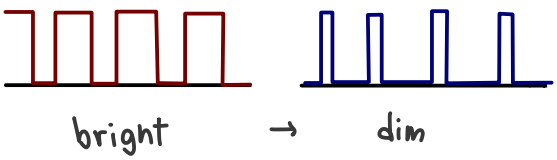
How many LEDs could you drive with the circuit in Figure 3 (this depends on the current limit of your power supply and the transistor)?

DC amp limit 1.5amp
BJT amp limit ~0.5amp
} $1.05/0.02 \simeq 52$ LEDs

PWM Control of Large Currents
Use a PWM output pin on the Arduino (e.g. pin 9) and show that you can control the brightness of the LED using the **Fade** sketch in the Examples folder

```
File > Examples > 1.Basics > Fade
```

How does the transistor control the brightness of the LED? (Check pin 9 output on the scope.)



Continuous transition between bright and dim intervals
↳ the transistor moves between longer/shorter active PWM intervals, giving the illusion of fading pulsating lighting

Keep the circuit you built! You will use it later in Part 3.

Name:_____ Date:_____

Part 2: Measuring Temperature

Thermocouples are devices for measuring temperature. They consist of two dissimilar metals, typically in the form of wires, joined together at one end. The junction between the dissimilar metals produces a voltage difference between the two wires that depends on the temperature of the junction. Thermocouples are separated into different types, depending on the metals used in the junction. We will use K-type thermocouple in this experiment, which have nickel-chromium and nickel-aluminum wires, and can be used from -200°C to 1250°C. In order to accurately read a thermocouple, you must precisely measure the voltage across the two wires, which changes only 41 µV/°C (for K type). Fortunately, there are pre-calibrated integrated circuits for reading thermocouples and even converting the voltage to temperature, such as the MAX6675.

In your Arduino kit, you will find an Adafruit MAX6675 breakout board and a K-type thermocouple. Plug the board into the Arduino Digital I/O Pins as indicated in Figure 4.

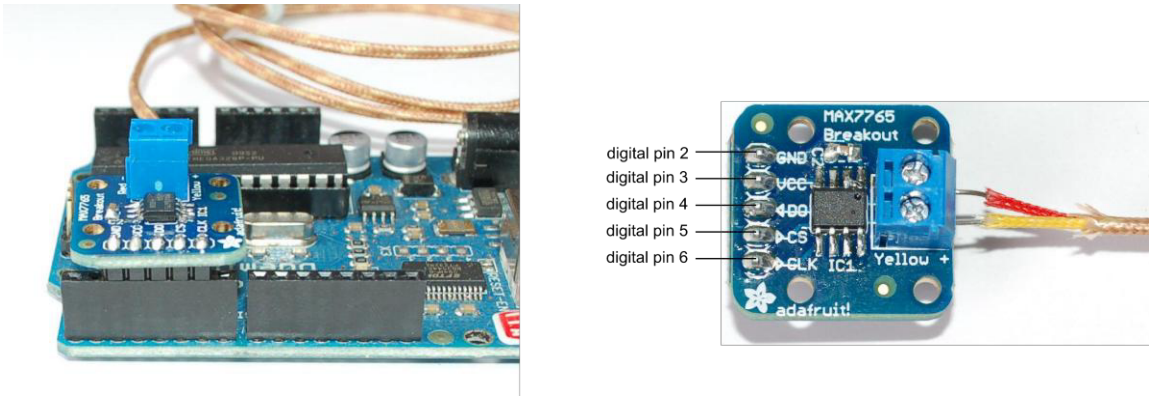


Figure 4. MAX6675 thermocouple amplifier breakout board.

To use this device, you will need to install the MAX6675 library, which you can find on Canvas. Once the library is installed (Sketch -> Include Library -> Add .Zip Library) run the **serialthermocouple** sketch in the `MAX6675` examples folder

```
File > Examples > MAX6675 > serialthermocouple
```

Display the temperature reading in the Serial Monitor and verify that the thermocouple is working. What temperature does the sensor record for the air temperature in the room?

76°F

What is the highest temperature you can reach holding the thermocouple junction in your hand?

97.25°F

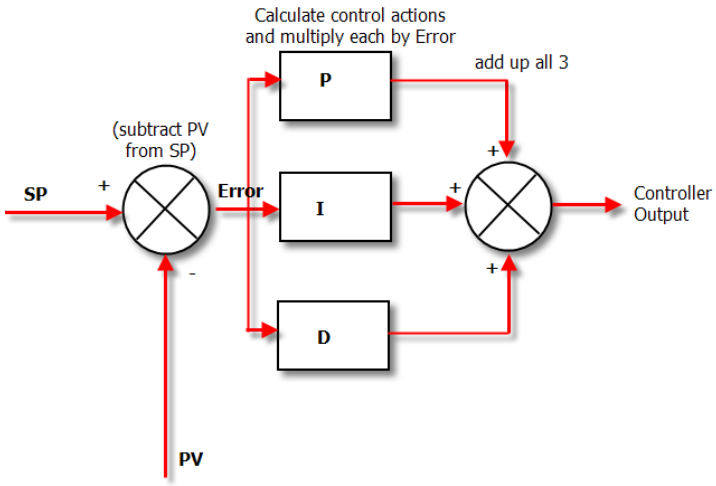
Name:_____ Date:_____

Part 3: Temperature Regulation

In this part you will combine the circuits from Parts 1 and 2 to regulate the temperature of a power resistor with a PID control loop.

In a research laboratory, a PID control loop might be part of an experiment that is sensitive to temperature changes because of precise alignment or a temperature-dependent chemical reaction. The PID algorithm is also widely used for other purposes, for example to control the position of the tip in a scanning tunneling microscope, the autofocus in a camera, the speed of an automobile (cruise control), or the stability of airplanes (autopilot). The algorithm is illustrated in Figure 5.

The PID algorithm is really very simple in operation. The PV (present value) is subtracted from the SP (setpoint) to create the Error. The error is simply multiplied by one, two or all of the calculated P, I and D actions (depending which ones are turned on). Then the resulting "error control actions" are added together and sent to the CO (controller output).



These 3 modes are used in different combinations:

- P – Sometimes used
- PI - Most often used
- PID – Sometimes used
- PD – rare as hen's teeth but can be useful for controlling servomotors.

Figure 5. Excerpt from **An Idiot's Guide to the PID Algorithm** by Finn Peacock.

Name:_____ Date:_____

The first step is to reconfigure the circuit from Part 1 to look like the one in Figure 6 below. Physically attach the tip of the thermocouple temperature sensor to the 130 Ω power resistor with a clothespin. **Use digital Pin 11 as the heater output**.

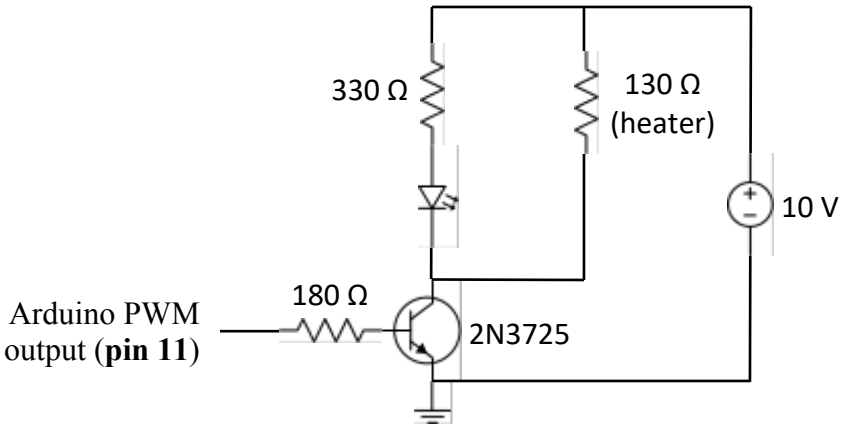


Figure 6. PID temperature control circuit.

The **PID_Sketch** to use for the PID control is available for download in Canvas. This sketch uses the Arduino Serial Monitor to display the setpoint and temperature history in graphical and text form. A screen shot of the Serial Monitor is shown in Figure 7.

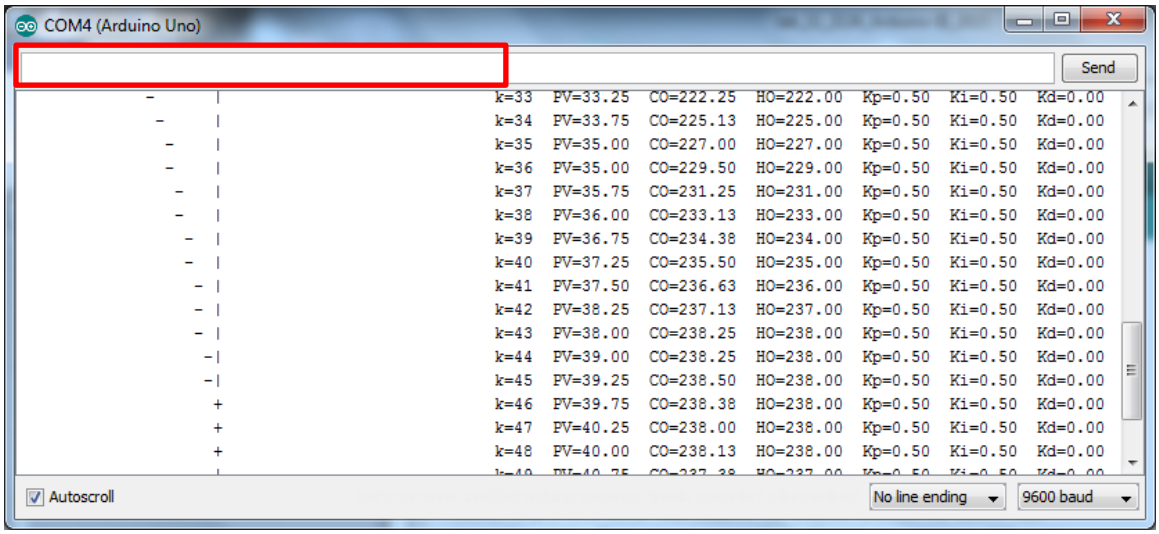


Figure 7. The output of PID_Sketch in Serial Monitor.

The temperature history is displayed as a strip chart that scrolls continuously. You can pause the PID loop any time and use the scroll bar to go back and review the data. The setpoint (SP) is plotted using the vertical bar "|" and the present value (PV) of the temperature is displayed using the horizontal dash "–". When these are equal the "+" symbol is used to show their overlap. You can select the data in the Serial Monitor, copy it using <Ctrl>+<c> and paste it into Excel for further analysis and plotting.

Name:_____ Date:_____

You can also control the PID loop and change its parameters interactively from the Serial Monitor. The Serial Monitor commands are listed in Table 1. These are entered on the Send line (see red box in Figure 7). Either hit the <Enter> key or click on the Send button to send the command to the Arduino. The numerical values xx.xx of the parameters are floating point numbers with a leading zero (e.g., 0.5) and any number of digits before or after the decimal point.

Table 1. Keyboard commands for controlling PID_Sketch from the Serial Monitor.

Keyboard Command	Function
<spacebar><Enter> or <spacebar> Send	Start the PID loop
<any key><Enter> or <any key> Send	Stop the PID loop
pxx.xx<Enter> or pxx.xx Send	Set K_p value
ixx.xx<Enter> or ixx.xx Send	Set K_i value
dxx.xx<Enter> or dxx.xx Send	Set K_d value

Note that **PID_Sketch** is incomplete, and you will need to use what you learnt about the PID control to complete the three lines of code in the `PID()` function that calculates the new control output value. The control output (CO) depends on the present error (P = proportional), the past error (I = integral), and the estimated future error (D = derivative), as shown in Figure 5. The gains of the error terms, K_p, K_i, and K_d, are the three adjustable parameters of the algorithm and you will need to find their optimum values to tune the PID algorithm for this specific application. This might seem like an easy task—after all there are only three parameters; however, thousands of books have been written on this topic and there is still no sure-fired recipe.

Write down the PID equation as you will use it in your sketch:

$$\Delta C = K_p(E_k - E_{k-1}) + K_i E_k \Delta t + \frac{K_d(E_k - E_{k-1}) - (E_{k-1} - E_{k-2})}{\Delta t}$$

Name:_____ Date:_____

Tuning the PID Algorithm

The goal of tuning a PID controller is to achieve fast response with good stability (i.e., small oscillation about the set point). Since these goals are diametrically opposed, the compromise goal is acceptable stability with medium response, which is illustrated in Figure 8. For a step change in setpoint, acceptable stability is when the undershoot that follows the first overshoot of the response is small, or barely observable.

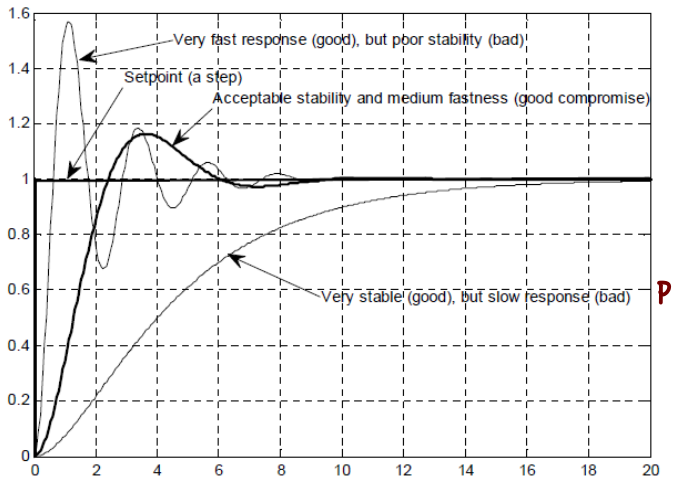


Figure 8. PID tuning goal
(http://home.hit.no/~hansha/documents/control/theory/tuning_pid_controller.pdf#page=2&zoom=auto,-71,558)

Before you begin tuning the PID controller it might help to look at some of the thermal dynamics of our system. Complete the PID() function in the **PID_Sketch** and upload it to the Arduino. Open the serial monitor and start the PID loop (hit space + Enter). If you do not see any output in the serial monitor, make sure the baud rate is set to 115200 bit/sec (instead of the default 9600) in the lower right corner of the serial monitor. Also set "No line ending" for the option next to the baud rate (see Fig. 7) if the display stops after the first line.

Next turn on the 10 V power supply. You will notice that the setpoint is set to 40 ºC, ~20 ºC above room temperature. However, because the gains K_p, K_i, and K_d are all set to zero, the heater does not turn on and the present value of the temperature does not change.

Now, looking at Figure 6, move the connection of the 180 Ω base resistor from Arduino Pin 11 to +5V. Instantly you will see the temperature rise. Get ready to yank the jumper wire from +5V when the present value reaches the setpoint, and connect it back to pin 11. Expand the Serial Monitor to see as much thermal history as possible.

With +5V to the base resistor, you are applying maximum power to the heater. How many steps (seconds) did it take to reach the setpoint from room temperature?

Max heating response: 10 seconds [s]

The heating curve is: _____ linear ✓ exponential

Name:_____ Date:_____

Optional: Can you derive the shape of the heating curve from thermodynamics?

Yes, $PV = nk_BT$

With the heater shut off you are applying maximum cooling. How many steps (seconds) did it take for the system to return to within 10% of room temperature after you shut off the heater?

Max cooling response: 50 seconds [s]

The cooling curve is: _____ linear ✓ exponential

Which response is quicker: _____ heating ✓ cooling

Optional: Can you derive the shape of the cooling curve from thermodynamics?

Yes, will I? No.

When tuning the PID, it is important to remember that the heater has a limit to the amount of power it can produce. In terms of PWM, the max heater output (HO) is represented by the value 255 and the heater is turned completely off when the PWM output is 0. So, it does not make sense for the controller output (CO) to be greater than 255 or less than 0.

Determining K_p

With K_i, and K_d still set to zero, set K_p to the following values and write down your observations. PV(t→∞) is the long-term average temperature of the resistor and ΔPV(t→∞) is the long-term stability or amplitude of oscillation in the temperature. After testing each value of K_p wait until the resistor has almost returned to room temperature.

K_p	K_i	CO_{max}	PV(t→∞)	ΔPV(t→∞)
1.0	0.0			
10.0	0.0			
100.0	0.0			
1000.0	0.0			

Name:_____ Date:_____

What does increasing the K_p parameter do?

What happens if the K_p parameter is too large?

Determining K_i

With K_p = 10.0, and K_d still set to zero, set K_i to the following values and write down your observations. $PV_{overshoot}$ is the amplitude of the first overshoot of the temperature above the setpoint (see Fig. 8). After testing each value of K_p wait until the resistor has almost returned to room temperature.

K_p	K_i	CO_{max}	$PV(t\rightarrow\infty)$	$\Delta PV(t\rightarrow\infty)$	$PV_{overshoot}$
10.0	0.1				
10.0	1.0				
10.0	10.0				
10.0	100.0				

What does increasing the K_i parameter do?

What happens if the K_i parameter is too large?

Final Try

K_p	K_i	CO_{max}	$PV(t\rightarrow\infty)$	ΔPV	$PV_{overshoot}$
100.0	1.0				

What optimal values of the K_p and K_i did you find?